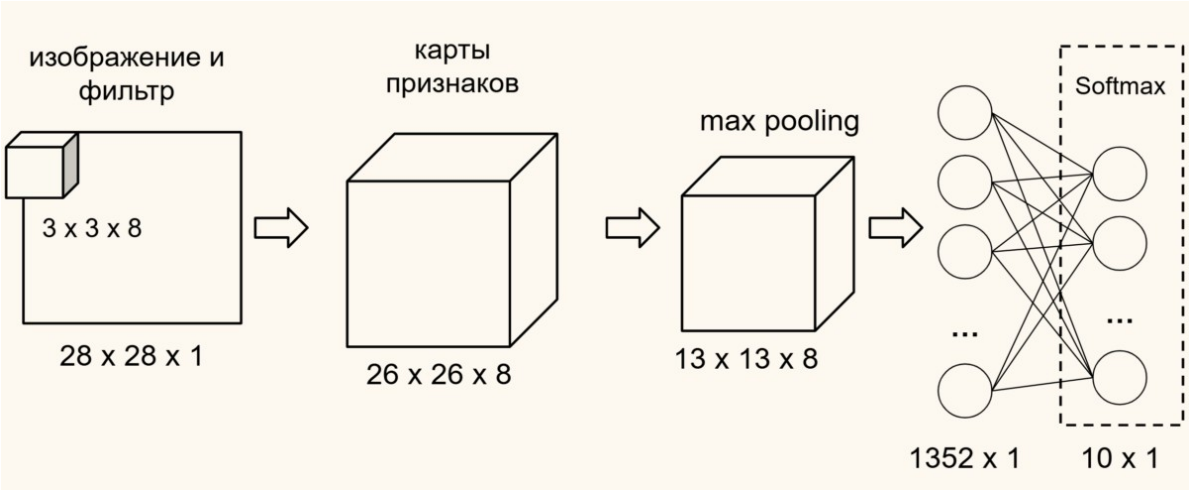
Свёрточная нейронная сеть. Часть 3

>

Создадим свёрточную нейросеть, используя встроенные функции Питона и библиотеку Numpy.

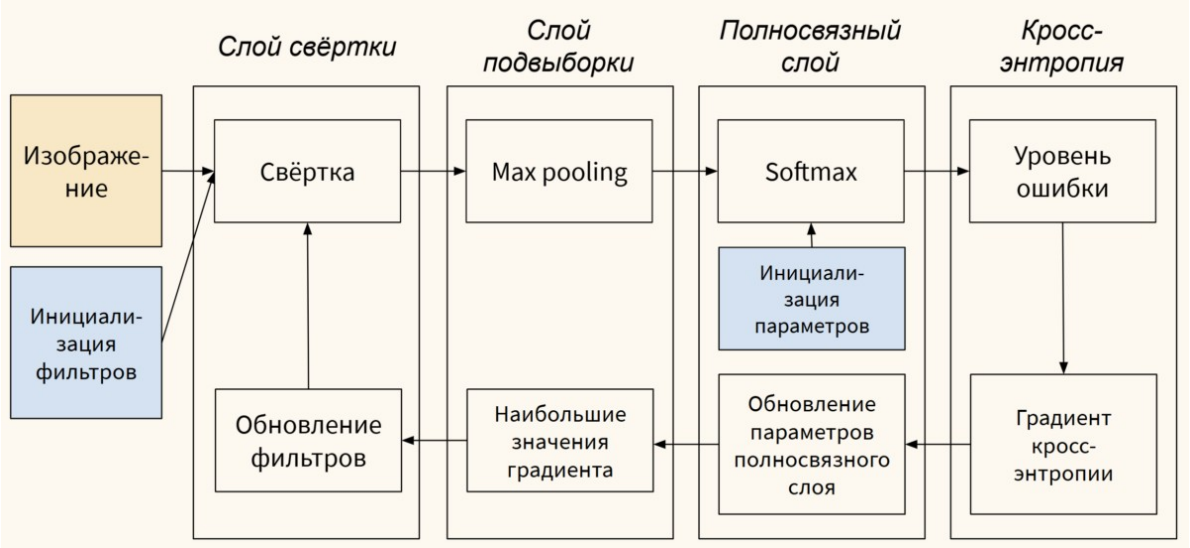
Архитектура сети

Будем использовать архитектуру, с которой мы познакомились во второй части с тем отличием, что для простоты откажемся от функции активации ReLU на свёрточном слое.



Общая схема

Приведем общую схему сети.



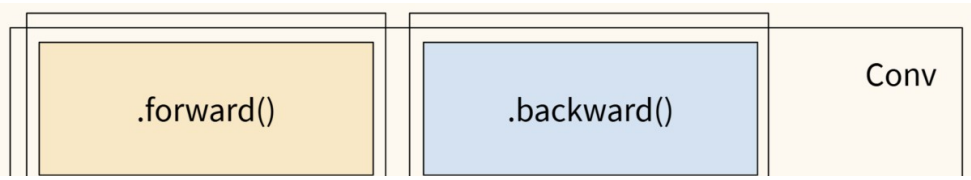
Получая на вход очередное изображение, мы инициализируем параметры фильтров и весов полносвязного слоя, а затем рассчитаем уровень ошибки при прямом распространении.

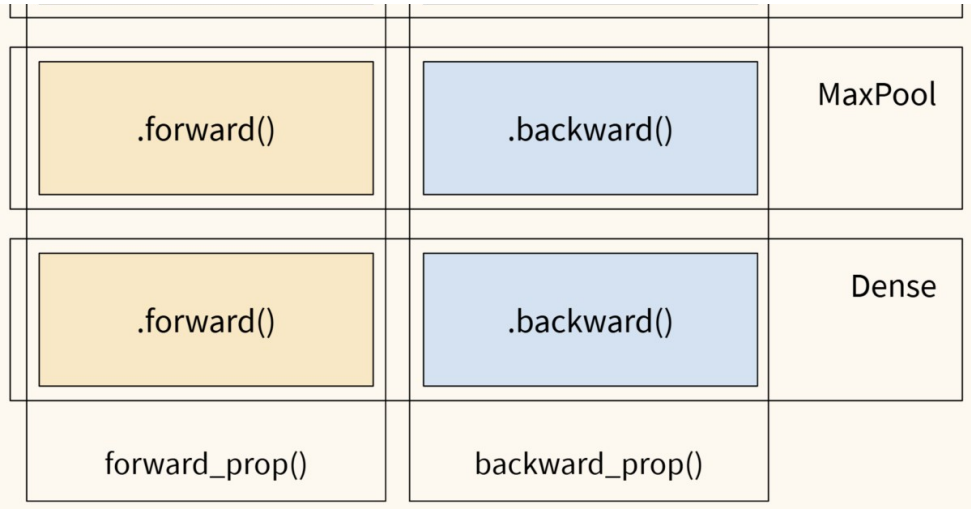
При обратном распространении мы последовательно обновим соответствующие параметры полносвязного и свёрточного слоев.

Организация кода

С точки зрения организации кода слой свёртки **Conv**, подвыборки **MaxPool** и полносвязный слой **Dense** мы поместим в отдельные классы, в которых будет два основных метода, **.forward()** и **.backward()**, для прямого и обратного распространения соответственно.

После этого мы создадим функции **forward_prop()** и **backward_prop()** для расчета уровня ошибки и распространения этой ошибки на обновляемые параметры.





Наконец, мы объявим функцию **fit_cnn()** для обучения нейросети, а затем **evaluate_cnn()** для оценки качества модели на тестовых данных.

Подготовим данные

Для ускорения обучения возьмем только первую тысячу изображений обучающих и тестовых данных.

```
import numpy as np

from tensorflow import keras
(X_train, y_train), (X_test, y_test) = keras.datasets.mnist.load_d

X_train = X_train[:1000] / 255 - 0.5
y_train = y_train[:1000]
X_test = X_test[:1000] / 255 - 0.5
y_test = y_test[:1000]
```

Начнем рассматривать каждый слой по отдельности.

Свёрточный слой

Вначале изучим каждую из функций свёрточного слоя, а затем объединим написанный код в класс Conv.

Вспомогательный метод

В первую очередь создадим вспомогательную функцию-генератор (в классе Conv мы превратим ее в метод), которая будет выдавать все возможные окна свёртки.

```
def sliding_windows(image):
  '''
  Создаёт все возможные окна свёртки на изображении
  с размером фильтра 3x3, размером шага 1 и отступом 0.
  '''
  h, w = image.shape

  # во внешнем цикле двигаемся по вертикали
  for i in range(h - 2):
    # во внутреннем - по горизонтали
    for j in range(w - 2):
      # каждый раз смещаемся на одно значение
      window = image[i:(i + 3), j:(j + 3)]
      # при каждом обращении к генератору выдаем массив значений 3x
      # индексы i и j текущего окна
      yield window, i, j
```

Проверим ее работу.

```
# выведем первые три окна и соответствующие индексы
for (idx, window) in enumerate(sliding_windows(X_train[0])):
  print(window)
  if idx == 2:
    break
```

```
(array([[-0.5, -0.5, -0.5],
       [-0.5, -0.5, -0.5],
       [-0.5, -0.5, -0.5]]), 0, 0)
(array([[-0.5, -0.5, -0.5],
       [-0.5, -0.5, -0.5],
       [-0.5, -0.5, -0.5]]), 0, 1)
(array([[-0.5, -0.5, -0.5],
       [-0.5, -0.5, -0.5],
       [-0.5, -0.5, -0.5]]), 0, 2)
```

Верхний левый угол изображения черный (значения пикселей равны нулю), поэтому после масштабирования мы получаем $-0{,}5$. Найдем общее количество таких окон.

```
# их должно быть 26 x 26 = 676
sum(1 for _ in sliding_windows(X_train[0]))
```

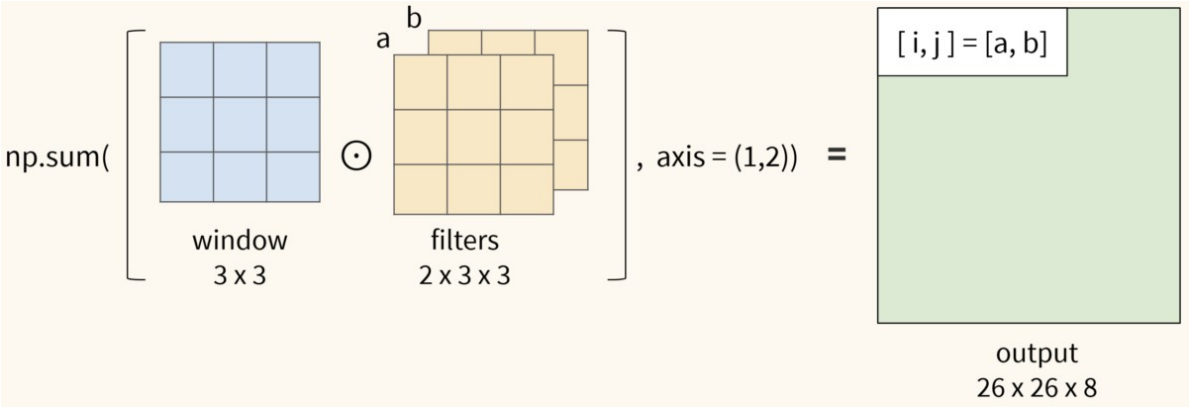
```
676
```

Прямое распространение

Выполним свёртку окон изображения с каждым из фильтров.

```
def forward(input, num_filters, filters):
  '''
  Выполняет свёртку (взаимную корреляцию) двумерных
  массивов Numpy: изображения и фильтров.
  Возвращает трехмерный массив Numpy с размерами (h, w, num_filter
  '''
  # зададим размерность карты признаков
  h, w = input.shape
  output = np.zeros((h - 2, w - 2, num_filters))

  # выполним свёртку
  for window, i, j in sliding_windows(input):
    # массив filters имеет размерность (num_filters, 3, 3),
    # поэтому сложение идет по измерениям 1 и 2
    output[i, j] = np.sum(window * filters, axis=(1,2))

  return output
```

Поясним предпоследнюю строку кода на отвлеченном примере.



Предположим, что у нас есть окно исходного изображения window размерностью 3 x 3 (его мы получаем из функции генератора) и два

(для простоты) фильтра 3 x 3.

```
wnd = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])

fltr = np.ones((2,3,3))
fltr
```

```
array([[[1., 1., 1.],
        [1., 1., 1.],
        [1., 1., 1.]],

       [[1., 1., 1.],
        [1., 1., 1.],
        [1., 1., 1.]]])
```

Поэлементное произведение позволяет умножить каждый из фильтров на значения окна.

```
wnd * fltr
```

```
array([[[1., 2., 3.],
        [4., 5., 6.],
        [7., 8., 9.]],

       [[1., 2., 3.],
        [4., 5., 6.],
        [7., 8., 9.]]])
```

Сложение по измерениям 1 и 2 (двум внутренним измерениям) позволит найти свёртку для каждого из двух фильтров.

```
np.sum(wnd * fltr, axis=(1,2))
```

```
array([45., 45.])
```

Результат свёртки (два значения по одному для каждого фильтра) мы поместим в массив `output` (с внешними измерениями 26 x 26) по координатам `[i,j]`.

Протестируем прямое распространение свёрточного слоя.

```
# создадим восемь фильтров
```

```
num_filters = 8
# размерностью 3 x 3
filters = np.random.randn(num_filters, 3, 3)
# посмотрим на первый инициализированный фильтр
filters[0]
```

```
array([[-1.00361337, -1.18814067, -1.35711705],
       [ 1.46267859, -1.71231877, -0.05552596],
       [-2.3648847 , -0.47920049, -1.25015735]])
```

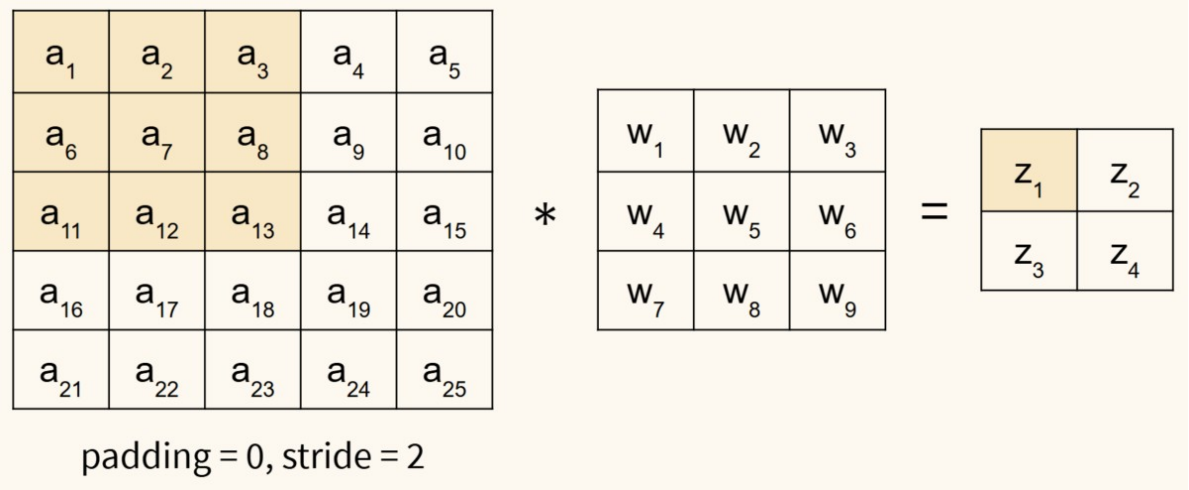
```
# проверим размерность свёртки, взяв первое изображение
conv_fwd = forward(X_train[0], num_filters, filters)
conv_fwd.shape
```

```
(26, 26, 8)
```

Перейдем к расчету градиента и обновлению параметров свёрточного слоя.

Обратное распространение

Вначале немного теории. Рассмотрим изображение 5 x 5 пикселей и фильтр размером 3 x 3. Произведем свёртку с нулевым отступом и размером шага два.



Таким образом, каждое из значений карты признаков можно найти через

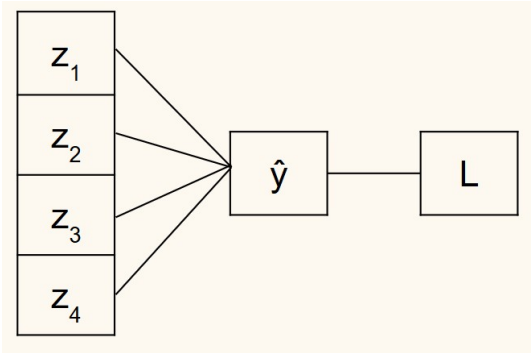
$$z_1 = w_1 \cdot a_1 + w_2 \cdot a_2 + w_3 \cdot a_3 + w_4 \cdot a_6 + \ldots + w_9 \cdot a_{13}$$

$$z_2 = w_1 \cdot a_3 + w_2 \cdot a_4 + w_3 \cdot a_5 + w_4 \cdot a_8 + \ldots + w_9 \cdot a_{15}$$

$$z_3 = w_1 \cdot a_{11} + w_2 \cdot a_{12} + w_3 \cdot a_{13} + w_4 \cdot a_{16} + \ldots + w_9 \cdot a_{23}$$

$$z_4 = w_1 \cdot a_{13} + w_2 \cdot a_{14} + w_3 \cdot a_{15} + w_4 \cdot a_{18} + \ldots + w_9 \cdot a_{25}$$

Теперь предположим, что после этого мы сразу «выпрямляем» карту признаков, делаем прогноз $\hat{y}$ и рассчитываем ошибку L.



Таким образом, для обновления параметров фильтра необходимо

$$w_i^* = w_i - \alpha \odot \frac{\partial L}{\partial w_i}$$

$$\begin{bmatrix} w_1^* & w_2^* & w_3^* \\ w_4^* & w_5^* & w_6^* \\ w_7^* & w_8^* & w_9^* \end{bmatrix} = \begin{bmatrix} w_1 & w_2 & w_3 \\ w_4 & w_5 & w_6 \\ w_7 & w_8 & w_9 \end{bmatrix} - \alpha \odot \begin{bmatrix} \partial L/\partial w_1 & \partial L/\partial w_2 & \partial L/\partial w_3 \\ \partial L/\partial w_4 & \partial L/\partial w_5 & \partial L/\partial w_6 \\ \partial L/\partial w_7 & \partial L/\partial w_8 & \partial L/\partial w_9 \end{bmatrix}$$

или, что то же самое,

$$w_i^* = w_i - \alpha \odot \left(\frac{\partial z_i}{\partial w_i} \odot \frac{\partial \hat{y}}{\partial z_i} \odot \frac{\partial L}{\partial \hat{y}} \right)$$

Выходящую из последующих слоев ошибку мы рассчитаем позднее, поэтому будем считать, что она нам известна и обозначим через

$$\frac{\partial L}{\partial z_i} = \frac{\partial \hat{y}}{\partial z_i} \odot \frac{\partial L}{\partial \hat{y}}$$

Таким образом,

$$w_i^* = w_i - \alpha \odot \left(\frac{\partial z_i}{\partial w_i} \odot \frac{\partial L}{\partial z_i} \right)$$

Найдем $\frac{\partial z_i}{\partial w_i}$. Заметим (см. расчеты $z_1, \ldots, z_4$ выше), что, например, изменение в w_1 влияет на *каждый* из $z_1, \ldots, z_4$. Значит,

$$\frac{\partial L}{\partial w_1} = \frac{\partial z_1}{\partial w_1}\frac{\partial L}{\partial z_1} + \frac{\partial z_2}{\partial w_1}\frac{\partial L}{\partial z_2} + \frac{\partial z_3}{\partial w_1}\frac{\partial L}{\partial z_3} + \frac{\partial z_4}{\partial w_1}\frac{\partial L}{\partial z_4}$$

Найдем $\frac{\partial z_1}{\partial w_1}$. Другими словами продифференцируем

$$z_1 = w_1 \cdot a_1 + w_2 \cdot a_2 + w_3 \cdot a_3 + w_4 \cdot a_6 + \ldots + w_9 \cdot a_{13}$$

относительно w_1, считая остальные веса константой.

$$\frac{\partial z_1}{\partial w_1} = (w_1 \cdot a_1 + w_2 \cdot a_2 + w_3 \cdot a_3 + w_4 \cdot a_6 + \ldots + w_9 \cdot a_{13})' = a_1$$

Аналогично,

$$\frac{\partial z_2}{\partial w_1} = (w_1 \cdot a_3 + w_2 \cdot a_4 + w_3 \cdot a_5 + w_4 \cdot a_8 + \ldots + w_9 \cdot a_{15})' = a_3$$
$$\frac{\partial z_3}{\partial w_1} = (w_1 \cdot a_{11} + w_2 \cdot a_{12} + w_3 \cdot a_{13} + w_4 \cdot a_{16} + \ldots + w_9 \cdot a_{23})' = a_{11}$$
$$\frac{\partial z_4}{\partial w_1} = (w_1 \cdot a_{13} + w_2 \cdot a_{14} + w_3 \cdot a_{15} + w_4 \cdot a_{18} + \ldots + w_9 \cdot a_{25})' = a_{13}$$

Таким образом,

$$\frac{\partial L}{\partial w_1} = a_1 \frac{\partial L}{\partial z_1} + a_3 \frac{\partial L}{\partial z_2} + a_{11} \frac{\partial L}{\partial z_3} + a_{13} \frac{\partial L}{\partial z_4}$$

Найдем частные производные относительно остальных весов.

$$\frac{\partial L}{\partial w_2} = a_2 \frac{\partial L}{\partial z_1} + a_4 \frac{\partial L}{\partial z_2} + a_{12} \frac{\partial L}{\partial z_3} + a_{14} \frac{\partial L}{\partial z_4}$$

$$\frac{\partial L}{\partial w_3} = a_3 \frac{\partial L}{\partial z_1} + a_5 \frac{\partial L}{\partial z_2} + a_{13} \frac{\partial L}{\partial z_3} + a_{15} \frac{\partial L}{\partial z_4}$$

$$\frac{\partial L}{\partial w_4} = a_6 \frac{\partial L}{\partial z_1} + a_8 \frac{\partial L}{\partial z_2} + a_{16} \frac{\partial L}{\partial z_3} + a_{18} \frac{\partial L}{\partial z_4}$$

$$\frac{\partial L}{\partial w_5} = a_7 \frac{\partial L}{\partial z_1} + a_9 \frac{\partial L}{\partial z_2} + a_{17} \frac{\partial L}{\partial z_3} + a_{19} \frac{\partial L}{\partial z_4}$$

$$\frac{\partial L}{\partial w_6} = a_8 \frac{\partial L}{\partial z_1} + a_{10} \frac{\partial L}{\partial z_2} + a_{18} \frac{\partial L}{\partial z_3} + a_{20} \frac{\partial L}{\partial z_4}$$

$$\frac{\partial L}{\partial w_7} = a_{11} \frac{\partial L}{\partial z_1} + a_{13} \frac{\partial L}{\partial z_2} + a_{21} \frac{\partial L}{\partial z_3} + a_{23} \frac{\partial L}{\partial z_4}$$

$$\frac{\partial L}{\partial w_8} = a_{12} \frac{\partial L}{\partial z_1} + a_{14} \frac{\partial L}{\partial z_2} + a_{22} \frac{\partial L}{\partial z_3} + a_{24} \frac{\partial L}{\partial z_4}$$

$$\frac{\partial L}{\partial w_9} = a_{13} \frac{\partial L}{\partial z_1} + a_{15} \frac{\partial L}{\partial z_2} + a_{23} \frac{\partial L}{\partial z_3} + a_{25} \frac{\partial L}{\partial z_4}$$

Обратим внимание, что пиксели при $\frac{\partial L}{\partial z_1}$ (выделены оранжевым) соответствуют первому из четырех окон свёртки. Аналогично, $\frac{\partial L}{\partial z_2}$ умножается на второе окно свёртки и т.д.

$$\begin{bmatrix} \partial L/\partial w_1 & \partial L/\partial w_2 & \partial L/\partial w_3 \\ \partial L/\partial w_4 & \partial L/\partial w_5 & \partial L/\partial w_6 \\ \partial L/\partial w_7 & \partial L/\partial w_8 & \partial L/\partial w_9 \end{bmatrix} = \begin{bmatrix} a_1 & a_2 & a_3 \\ a_6 & a_7 & a_8 \\ a_{11} & a_{12} & a_{13} \end{bmatrix} \odot \partial L/\partial z_1 + \ldots$$

Перейдем к коду. Обозначим через `d_L_d_out` распространение ошибки от последующих слоев. В формулах выше мы обозначали его через $\frac{\partial L}{\partial z_i}$.

```
def backward(d_L_d_out, filters, learning_rate):
  '''
  Выполняет обновление параметров фильтров свёрточного слоя.
  Принимает массив 26 x 26 x 8 и возвращает 8 x 3 x 3.
  '''
  # зададим размеры градиента фильтров
  d_L_d_filters = np.zeros(filters.shape)

  # найдем окна свёртки на изображении
  for window, i, j in sliding_windows(X_train[0]):
    # для каждого фильтра
```

```
    for f in range(num_filters):
      # умножим соответствующее окно на ошибку последующих слоев
      # и сложим получившиеся произведения
      d_L_d_filters[f] += window * d_L_d_out[i, j, f]

  # обновим параметры фильтров
  filters -= learning_rate * d_L_d_filters

  return filters
```

```
# протестируем обновление параметров фильтров
conv_bwd = backward(conv_fwd, filters, 0.005)
conv_bwd.shape
```

```
(8, 3, 3)
```

Класс Conv

Поместим созданный для свёрточного слоя код в класс Conv.

```
class Conv:

  def __init__(self, num_filters):

    self.num_filters = num_filters
    self.filters = np.random.randn(num_filters, 3, 3)

  def sliding_windows(self, image):

    h, w = image.shape

    for i in range(h - 2):
      for j in range(w - 2):
        window = image[i:(i + 3), j:(j + 3)]
        yield window, i, j

  def forward(self, input):

    # запомним последнее переданное изображение,
    # чтобы использовать его в методе .backward()
    self.last_input = input

    h, w = input.shape
    output = np.zeros((h - 2, w - 2, self.num_filters))
```

```
    for window, i, j in self.sliding_windows(input):
      output[i, j] = np.sum(window * self.filters, axis=(1,2))

    return output

  def backward(self, d_L_d_out, learning_rate):

    d_L_d_filters = np.zeros(self.filters.shape)

    for window, i, j in self.sliding_windows(self.last_input):
      for f in range(self.num_filters):
        d_L_d_filters[f] += window * d_L_d_out[i, j, f]

    self.filters -= learning_rate * d_L_d_filters

    # так как это первый слой нейросети, дальше ошибку
    # распространять не нужно
    return None
```

Перейдем к рассмотрению слоя подвыборки по максимальному значению.

Max pooling

Вспомогательный метод

В классе MaxPool используем вспомогательный метод-генератор, который будет создавать для карты признаков все возможные окна размером 2 x 2.

```
def maxpool_windows(self, feature_map):
  '''
  Создаёт все возможные max pooling окна для одной карты признаков
  '''
  # найдем высоту и ширину карты признаков
  h, w, _ = feature_map.shape

  # уменьшим эти размеры в два раза
  for i in range(h // 2):
    for j in range(w // 2):
```

```
        window = feature_map[(i * 2):(i * 2 + 2), (j * 2):(j * 2 + 2
        # при каждом обращении к генератору будем выдавать по одному
        # окну max pooling и соответствующие индексы
        yield window, i, j
```

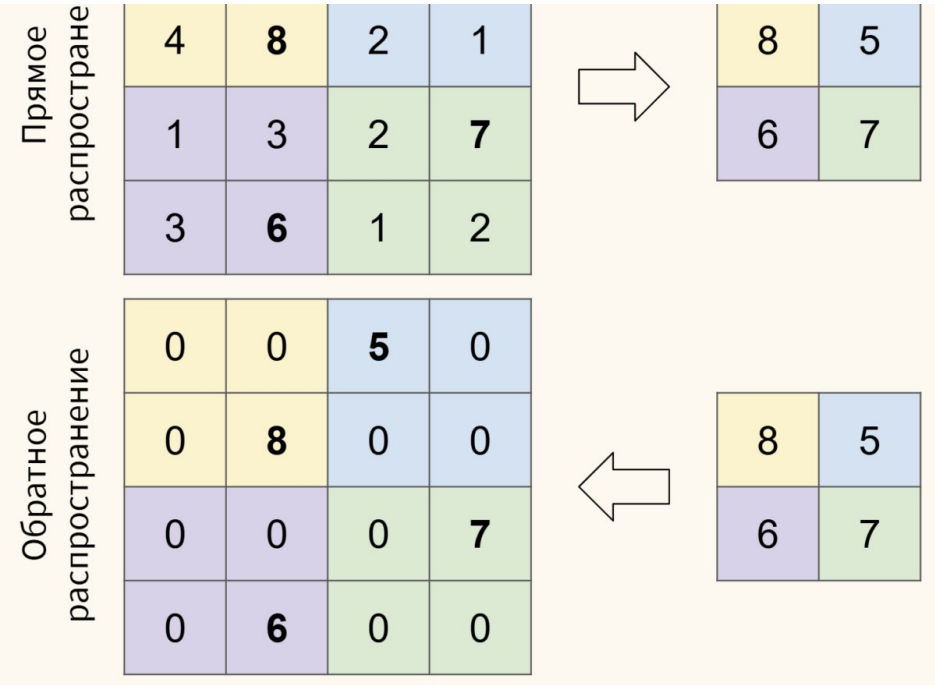
Прямое распространение

После этого, используя метод **.forward()** пройдемся по каждому из окон и внутри каждого выберем наибольшее значение с помощью функции **np.amax()**.

```
def forward(self, input):
  '''
  Выполняет подвыборку по максимальному значению.
  Принимает трехмерный массив Numpy с размерами (h, w, num_filters
  Возвращает трехмерный массив Numpy с размерами (h/2, w/2, num_fi
  '''
  # запомним последнюю карту признаков,
  # чтобы использовать ее в методе .backward()
  self.last_input = input

  h, w, num_filters = input.shape
  output = np.zeros((h // 2, w // 2, num_filters))

  for window, i, j in self.maxpool_windows(input):
    output[i, j] = np.amax(window, axis=(0,1))

  return output
```

Обратное распространение

Если при прямом распространении мы сокращаем размер карты признаков в два раза, то при обратном нам нужно его восстановить таким образом, чтобы вернуть максимальные значения на свои места, а остальные значения заполнить нулями.



Идея заключается в том, что (1) изменение **немаксимальных значений**, поскольку они «исчезают» при прямом распространении, никак не влияет на уровень ошибки и соответственно их градиент равен нулю.

$$\frac{\partial L}{\partial maxpool} = \frac{\partial out}{\partial maxpool} \odot \frac{\partial L}{\partial out} = (maxpool \cdot 0) \odot \frac{\partial L}{\partial out}$$

Если же (2) значение было **максимальным**, то оно влияет на ошибку линейно, а значит производная равна самому этому значению.

$$\frac{\partial L}{\partial maxpool} = \frac{\partial out}{\partial maxpool} \odot \frac{\partial L}{\partial out} = (maxpool \cdot 1) \odot \frac{\partial L}{\partial out}$$

В самом деле, эффект максимального значения (например, 8) можно выразить через линейную зависимость $8x$, поскольку увеличение значения в два раза приведет к увеличению эффекта на ошибку в $8 \cdot 2 = 16$. Несложно увидеть, что производная будет равна $8 \cdot 1 = 8$.

Поясним, что $\frac{\partial L}{\partial out}$ (в коде ниже `d_L_d_out`) — это распространение ошибки вплоть до слоя подвыборки, $\frac{\partial out}{\partial maxpool}$ (в коде `d_L_d_input`) — распространение ошибки самого этого слоя.

```
def backward(self, d_L_d_out):
```

```
    '''
    Выполняет обратное распространение ошибки слоя подвыборки.
    Принимает массив Numpy с размерами (h/2, w/2, num_filters).
    Возвращает массив Numpy с размерами (h, w, num_filters)
    '''
    # создадим массив из нулей с размерностью карты признаков
    d_L_d_input = np.zeros(self.last_input.shape)

    # берем окна карты признаков из метода .forward()
    for window, i, j in self.maxpool_windows(self.last_input):
      h, w, f = window.shape
      # находим максимальные значения по первым двум измерениям
      amax = np.amax(window, axis=(0,1))

      # с помощью индексов проходимся по значениям карты признаков
      for i2 in range(h):
        for j2 in range(w):
          for f2 in range(f):
            # если значение является максимальным,
            if window[i2, j2, f2] == amax[f2]:
              # скопируем его в d_L_d_input
              d_L_d_input[i * 2 + i2, j * 2 + j2, f2] = d_L_d_out[i,

    return d_L_d_input
```

Очевидно, обучаемых параметров здесь не будет.

Полносвязный слой и кросс-энтропия

Прямое распространение

Инициализируем параметры полносвязного слоя.

```
def __init__(self, flatten_nodes, softmax_nodes):
  # инициализируем веса полносвязного слоя 1352 x 10
  self.weights = np.random.randn(flatten_nodes, softmax_nodes) / f
```

Объявим вспомогательный метод **softmax()**.

```
def softmax(self, z):
```

```
    z = z - np.max(z, axis = 0, keepdims = True)
    numerator = np.exp(z)
    denominator = np.sum(numerator, axis = 0, keepdims = True)
    softmax = numerator / denominator
    return softmax
```

Пропишем прямое распространение.

```
def forward(self, input):
    '''
    Выполняет прямое распространение полносвязного слоя.
    Принимает трехмерный массив слоя подвыборки.
    Возвращает одномерный массив с вероятностями для каждого класса.
    '''
    # запомним размерность после слоя max pool
    self.last_input_shape = input.shape
    # "выровняем" значения
    input = input.flatten()
    # запомним их для метода .backward()
    self.last_input = input
    # умножим выровненные значения на веса
    z = np.dot(input, self.weights)
    self.last_z = z
    # найдем вероятности каждого класса с помощью softmax
    return self.softmax(z)
```

Кросс-энтропия

Найдем уровень ошибки через

$$L = -\ln(p_c),$$

где p_c — вероятность правильного класса для данного изображения. Поясним, что формула отличается от использовавшейся ранее,

$$L(y_{ohe}, softmax) = -\sum y_{ohe} \cdot log(softmax)$$

поскольку предсказываемая цифра (в коде `label`) является одновременно индексом класса, и нет необходимости использовать one-hot encoding.

```
def cross_entropy_loss(out, label):
  loss = -np.log(out[label])
  return loss
```

Градиент кросс-энтропии относительно softmax можно найти по формуле.

$$\frac{\partial L}{\partial softmax} = \begin{cases} 0 & \text{если } i \neq c \\ -\frac{1}{p_i} & \text{если } i = c \end{cases}$$

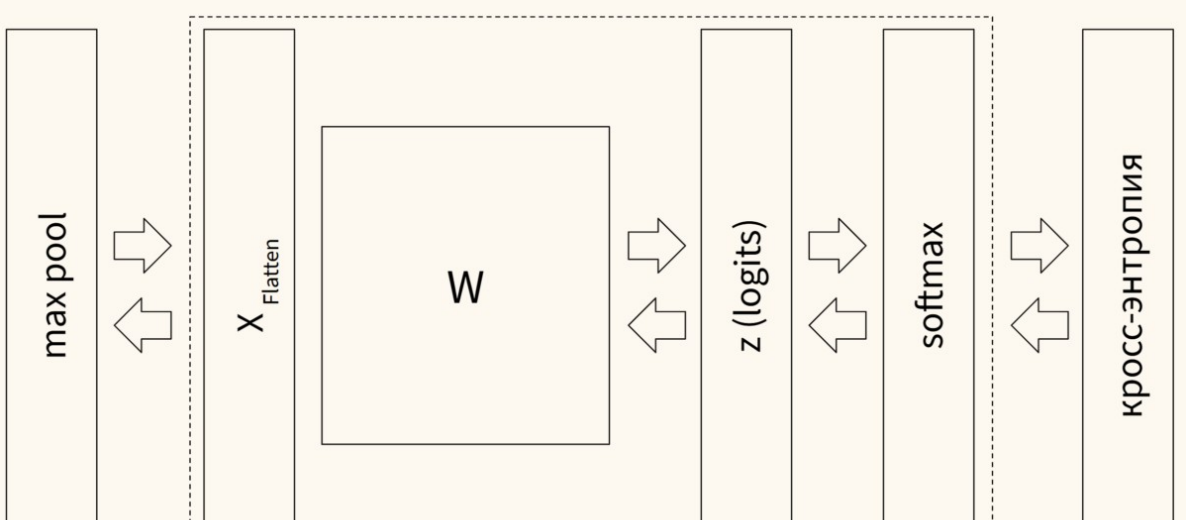
```
def cross_entropy_gradient(out, label, softmax_nodes):
  gradient = np.zeros(softmax_nodes)
  gradient[label] = -1 / out[label]
  return gradient
```

Заметим, что в данном случае лишь один из градиентов будет ненулевым.

Обратное распространение

Найдем производные функции softmax вначале

- относительно результатов умножения матрицы весов на значения z слоя flatten (так называемых **логитов**, logits); затем
- относительно весов W полносвязного слоя (их нужно будет обновить) и значений X_{flatten}, входящих в полносвязный слой (чтобы дальше распространять ошибку на слой подвыборки).



Производная функции softmax

Обозначим функцию softmax для предсказываемого класса c через $\varphi(c)$

$$\varphi(c) = \frac{e^{z_c}}{\sum_i e^{z_i}} = \frac{e^{z_c}}{S} = e^{z_c} S^{-1},$$

где $z = X_{\text{flatten}} \cdot W$.

Частная производная относительно класса $k \neq c$, будет равна

$$\frac{\partial \varphi(c)}{\partial z_k} = \frac{\partial \varphi(c)}{\partial S} \odot \frac{\partial S}{\partial z_k} = -e^{z_c} S^{-2} \left(\frac{\partial S}{\partial z_k} \right) = -e^{z_c} S^{-2} e^{z_k} = \frac{-e^{z_c} e^{z_k}}{S^2}$$

Для $k = c$,

$$\frac{\partial \varphi(c)}{\partial z_c} = \frac{\left(S e^{z_c} - e^{z_c} \left(\frac{\partial S}{\partial z_c} \right) \right)}{S^2} = \frac{S e^{z_c} - e^{z_c} e^{z_c}}{S^2} = \frac{e^{z_c}(S - e^{z_c})}{S^2}$$

Таким образом,

$$\frac{\partial \varphi(c)}{\partial z} = \begin{cases} \frac{-e^{z_c} e^{z_k}}{S^2} & \text{если } k \neq c \\ \frac{e^{z_c}(S - e^{z_c})}{S^2} & \text{если } k = c \end{cases}$$

Производная весов и входящих значений

Еще раз вспомним, что $z = X_{\text{flatten}} \cdot W$. Значит,

$$\frac{\partial z}{\partial W} = X_{\text{flatten}}, \qquad \frac{\partial z}{\partial X_{\text{flatten}}} = W$$

Напишем соответствующий метод.

```
def backward(self, d_L_d_out, learing_rate):
  '''
  Выполняет обратное распространение полносвязного слоя.
  Принимает градиент кросс-энтропии и коэффициент скорости обучени
  Возвращает массив с размерностью результата слоя подвыборки.
  '''
  # только один из градиентов будет ненулевым
  for i, gradient in enumerate(d_L_d_out):
    if gradient == 0:
      continue

    # e^z
    z_exp = np.exp(self.last_z)
    # сумма всех e^z
    S = np.sum(z_exp)

    # градиент softmax относительно z
    d_out_d_z = -z_exp[i] * z_exp / (S ** 2)
    d_out_d_z[i] = z_exp[i] * (S - z_exp[i]) / (S ** 2)

    # градиент относительно весов и входящих значений
    d_z_d_w = self.last_input
    d_z_d_inputs = self.weights

    # распространение ошибки до функции softmax
    d_L_d_z = gradient * d_out_d_z

    # распространение ошибки до весов и входящих значений
    d_L_d_w = d_z_d_w[np.newaxis].T @ d_L_d_z[np.newaxis]
    d_L_d_inputs = d_z_d_inputs @ d_L_d_z

    # обновление весов
    self.weights -= learing_rate * d_L_d_w

    # градиент входящих значений с размерностью слоя подвыборки
    return d_L_d_inputs.reshape(self.last_input_shape)
```

Функции forward_prop() и backward_prop()

Напишем функции для прямого и обратного распространения через все слои нейросети.

```
def forward_prop(image, label):
  '''
  Выполняет прямое распространение свёрточной сети.
  Принимает одно изображение (двумерный массив) и значение класса.
  Рассчитывает ошибку и accuracy.
  '''
  out = conv.forward(image)
  out = pool.forward(out)
  out = dense.forward(out)

  loss = cross_entropy_loss(out, label)
  accuracy = 1 if np.argmax(out) == label else 0

  return out, loss, accuracy

def backward_prop(out, label, learing_rate):
  '''
  Выполняет обратное распространение свёрточной сети.
  Принимает градиент кросс-энтропии, значение класса и коэффициент
  Обновляет веса. Возвращает None.
  '''
  gradient = cross_entropy_gradient(out, label, softmax_nodes)
  gradient = dense.backward(gradient, learing_rate)
  gradient = pool.backward(gradient)
  conv.backward(gradient, learing_rate)

  return None
```

Обучение модели

Создадим объекты классов слоев модели и зададим соответствующие гиперпараметры.

```
n_filters, flatten_nodes, softmax_nodes = 8, 13 * 13 * 8, 10

conv = Conv(n_filters)
pool = MaxPool()
dense = Dense(flatten_nodes, softmax_nodes)
```

Объявим функции для обучения и оценки качества модели.

```
def fit_cnn(X, y, epochs=3, learing_rate=.005):
```

```
  for epoch in range(epochs):

    idx_shuffle = np.random.permutation(len(X))
    X = X[idx_shuffle]
    y = y[idx_shuffle]

    loss_total, acc_total = 0, 0

    for i, (image, label) in enumerate(zip(X,y)):

      out, loss, acc = forward_prop(image, label)
      backward_prop(out, label, learing_rate)

      loss_total += loss
      acc_total += acc

    print('Эпоха', (epoch + 1))
    print('Ошибка:', np.round(loss_total/len(X), 3))
    print('Accuracy:', np.round(acc_total/len(X), 3))
    print('-----------------------')

def evaluate_cnn(X, y):
  loss_total, acc_total = 0, 0

  for image, label in zip(X, y):
    _, loss, acc = forward_prop(image, label)
    loss_total += loss
    acc_total += acc

  print('Ошибка:', np.round(loss_total / len(X), 2))
  print('Accuracy:', np.round(acc_total / len(X), 2))
```

Обучим модель и оценим ее качество.

```
fit_cnn(X_train, y_train)
```

```
Эпоха 1
Ошибка: 0.858
Accuracy: 0.767
-----------------------
Эпоха 2
Ошибка: 0.302
Accuracy: 0.91
-----------------------
Эпоха 3
```

```
Ошибка: 0.19
Accuracy: 0.95
-----------------------
```

```
evaluate_cnn(X_test, y_test)
```

```
Ошибка: 0.44
Accuracy: 0.86
```